CS791 Assignment4 [Team: MSD]

Madhav Kotecha (24M0833)*
Smita Gautam (24M0845)*
Deeksha Koul (24D0365)*
*Equal contribution

November 5th, 2025

1 Task 1: Exploring Kernel and Acquisition functions

Setup

Optimized the following hyperparameters:

- Hidden Layer Size: {100, 200, 300, 400, 500}
- Training Epochs: [1, 10]
- Learning Rate: $[10^{-5}, 10^{-1}]$
- Batch Size: {16, 32, 64, 128, 256}
- Dropout Rate: [0, 0.5]
- Weight Decay: $[10^{-6}, 10^{-2}]$

Total budget $N = 25$ runs with $N_{warmup} = 10$ random samples. Six configurations were tested (3 kernels $\times$ 2 acquisition functions).

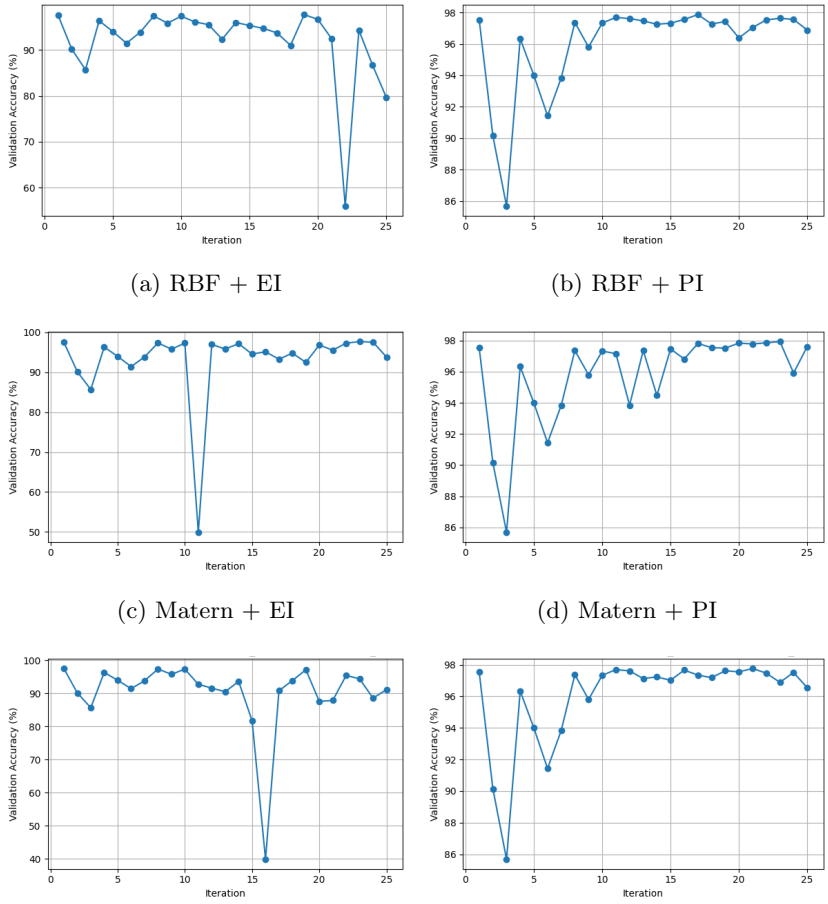
For this task, six configurations were tested using combinations of three kernels (RBF, Matern and Rational Quadratic) and two acquisition functions (Expected Improvement (EI) and Probability of Improvement (PI)). The goal was to optimize the neural network hyperparameters to maximize test accuracy. Table 1 summarizes the final optimized hyperparameters and resulting accuracies for $N = 25$ function evaluations.

Results

Figures below show validation accuracy progression across N = 25 iterations for each kernel–acquisition pair.

Kernel	Acquisition	Hidden Size	Epochs	Learning Rate	Batch Size	Dropout Rate	Weight Decay	Accuracy (%)
RBF	EI	400	10	0.0037	256	0.2931	$7.3686e^{-05}$	97.85
RBF	PI	500	8	0.0008	64	0.0909	$1.0252e^{-06}$	97.87
Matern	EI	300	9	0.0086	32	0.4104	$2.8694e^{-05}$	97.94
Matern	PI	400	9	0.0008	32	0.4305	$1.2242e^{-05}$	98.06
Rational Quadratic	EI	400	8	0.0025	32	0.0780	1.7074-06	97.85
Rational Quadratic	PI	400	7	0.0012	64	0.2741	$1.3461e^{-06}$	97.73

Table 1: Final optimized hyperparameters and test accuracy for each kernel–acquisition combination (N = 25)



(a) RBF + EI

(b) RBF + PI

(c) Matern + EI

(d) Matern + PI

(e) Rational Quadratic + EI

(f) Rational Quadratic + PI

Figure 1: Validation accuracy progression for NN ($N = 25$).

Observations

- Validation curves were mostly stable after iteration 10, showing that BO quickly found good regions.

- Matern kernel showed the most consistent improvement and the highest test accuracy (98.06% with PI).
- Rational Quadratic showed a bit more ups and downs, suggesting the search was less stable across iterations.
- PI is generally smoother(there are not many dips, whereas in EI the dip can go as low as 48% in RQ + EI setup)
- Best hyperparameters for Matern + PI:
  - hidden_size = 400, training_epochs = 9
  - lr = 0.0008333, batch_size = 32
  - dropout_rate = 0.4305, weight_decay = $1.22e^{-05}$

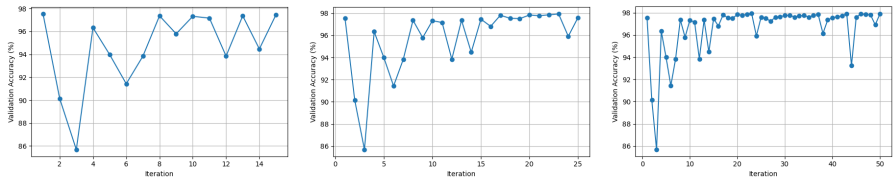
Conclusion: Overall, the validation accuracy trends show that all kernel and acquisition pairs stabilize with above 95% accuracy after a few iterations(10). The Matern kernel with PI shows the most consistent improvement and highest peak accuracy, while RBF and Rational Quadratic perform comparably but with a few noisy dips.

Effect of Budget (N)

After identifying **Matern + PI** as the best-performing configuration, the impact of the evaluation budget N was analyzed using $N = \{15, 25, 50\}$. Table 2 lists the optimized hyperparameters and final test accuracies for each budget.

N	Hidden Size	Epochs	Learning Rate	Batch Size	Dropout Rate	Weight Decay	Accuracy (%)
15	400	8	0.0025	32	0.0780	$1.71e^{-06}$	97.78
25	400	9	0.0008	32	0.4305	$1.22e^{-05}$	98.06
50	400	9	0.0008	32	0.4305	$1.22e^{-05}$	98.07

Table 2: Effect of varying budget (N) on optimized hyperparameters and accuracy (Matern + PI)



(a) $N = 15$ (b) $N = 25$ (c) $N = 50$

Figure 2: Validation accuracy progression for Matern + PI across different evaluation budgets N.

Analysis: Moving N from 15 to 25 gave a small boost in accuracy(97.78% to 98.06%), but beyond that, we didnt see any marginal change. At N=50, the results were almost the same as at N=25, suggesting the optimizer had already found a near-optimal configuration.

Run main.py to execute the entire BO process

```
python3 main.py --model_type nn \
--acquisition_function pi --kernel matern \
--max_budget 25 --init_points 10
```

2 Task 2: Exploring BO for a different architecture

Setup

Optimized the following hyperparameters:

- Training Epochs: [5, 20]
- Learning Rate: $[10^{-4}, 10^{-2}]$
- Batch Size: {32, 64, 128, 256}
- Optimizer: {Adam, RMSProp}

Other than that, we have fixed the remaining parameters:

- Hidden Layer Size: 256
- Dropout Rate: 0
- Weight Decay: 10^{-4}

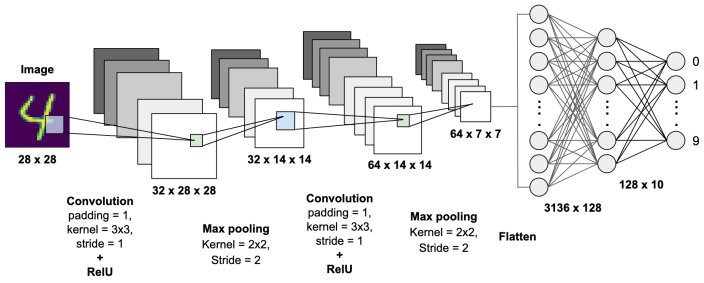


Figure 3: CNN Architecture used in Task 2. Source: Arun Pandian R [2020]

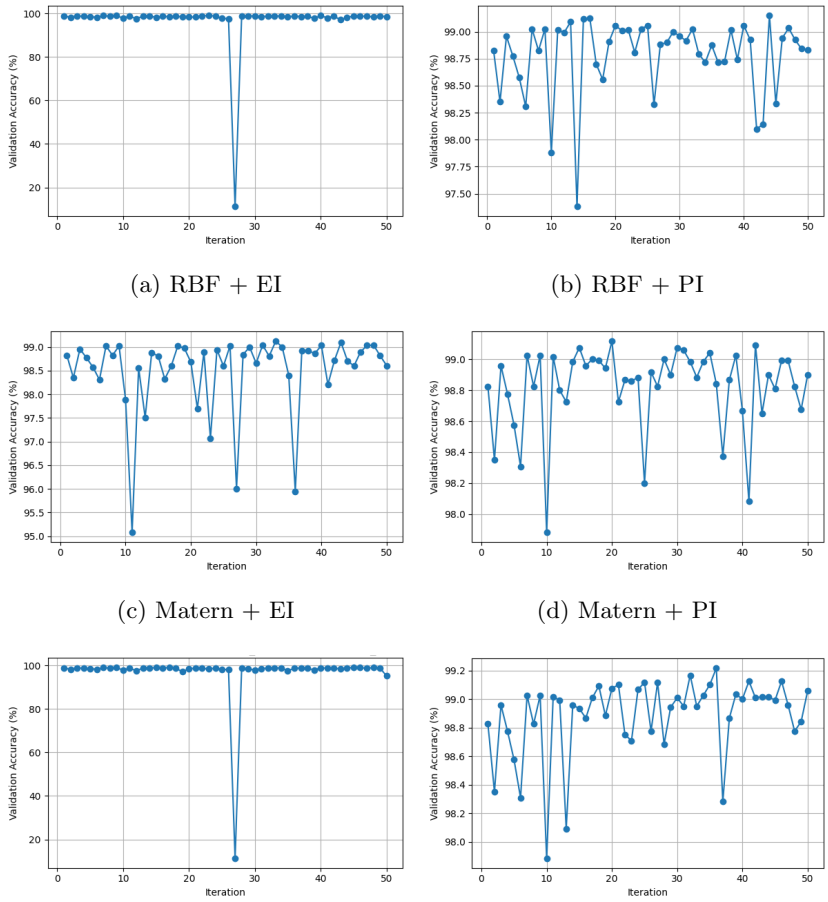
Total budget $N = 50$ runs with $N_{warmup} = 10$ random samples. Six configurations were tested (3 kernels $\times$ 2 acquisition functions).

Same as Task-1, for this task, six configurations were tested using combinations of three kernels (RBF, Matern and Rational Quadratic) and two acquisition functions (Expected Improvement (EI) and Probability of Improvement (PI)). The goal was to optimize the convolutional neural network hyperparameters to maximize test accuracy. Table 3 summarizes the final optimized hyperparameters and resulting accuracies for $N = 50$ function evaluations.

Kernel	Acquisition	Epochs	Learning Rate	Batch Size	Optimizer	Accuracy (%)
RBF	EI	20	0.0029	256	RMSProp	98.91
RBF	PI	20	0.0011	32	RMSProp	99.18
Matern	EI	20	0.0007	128	RMSProp	98.89
Matern	PI	16	0.0007	128	RMSProp	99.05
Rational Quadratic	EI	20	0.0006	32	RMSProp	99.17
Rational Quadratic	PI	20	0.0013	64	RMSProp	99.35

Table 3: Final optimized hyperparameters and test accuracy for each kernel–acquisition combination (N = 50)

Results



(a) RBF + EI

(b) RBF + PI

(c) Matern + EI

(d) Matern + PI

(e) Rational Quadratic + EI

(f) Rational Quadratic + PI

Figure 4: Validation accuracy progression for CNN ($N = 50$).

Figures above show validation accuracy progression across N = 50 iterations for each kernel–acquisition pair.

Observations

- Best hyperparameters for Rational Quadratic + PI:
  - training_epochs = 20
  - lr = 0.0013635, batch_size = 64
  - dropout_rate = RMSProp
- With CNN, all kernel-acquistion pairs(except RBF+EI having a sudden dip around 28th iteration) shows good comparable performance, with accuracy around 98-99%.
- The PI acquisition was generally smoother, which we observed in the simple NN case as well.
- Overall, the results are steadier than with the simple NN, suggesting the CNN can handle hyperparameter changes more robustly.

Architecture breakdown

Here is the breakdown of the calculation for the Convolutional NN architecture with a hidden_size=256:

1. **Convolutional Layers**
   - `conv1_1` ($1 \to 32$, 3x3 kernel): 320 parameters
   - `conv1_2` ($32 \to 32$, 3x3 kernel): 9,248 parameters
   - `conv2_1` ($32 \to 64$, 3x3 kernel): 18,496 parameters
   - `conv2_2` ($64 \to 64$, 3x3 kernel): 36,928 parameters
   - **Subtotal Conv: 64,992**
2. **Fully Connected Layers**
   - `flattened_size`: $64 \times 7 \times 7 = 3,136$
   - `fc1` (3,136 $\to$ 256): $(3,136 \times 256) + 256 = 803,072$ parameters
   - `fc2` ($256 \to 10$): $(256 \times 10) + 10 = 2,570$ parameters
   - **Subtotal FC: 805,642**

Total: $64,992$ (Conv) + $805,642$ (FC) = $\mathbf{870,634}$ parameters

Run main.py to execute the entire BO process

```
python3 main.py --model_type cnn \
--acquisition_function pi --kernel rational_quadratic \
--max_budget 50 --init_points 10
```

References

Arun Pandian R. Digit recognition using cnn. https://www.kaggle.com/code/arunrk7/digit-recognition-using-cnn-99-accuracy, 2020.